

ชื่อกลุ่ม / เลขกลุ่ม

กลุ่ม 4

หน้าต่านี้ยังดาวน์โหลดไม่ได้

สมาชิกในกลุ่ม

+ เพิ่มสมาชิก

จีระพงษ์ คาคไธสง

680112418029

x

Data Communication – Lab Worksheet

รู้จักเครื่องของตัวเองผ่าน SSH

แบบฝึกหัดภาคปฏิบัติ 4 ส่วน: พื้นฐาน Linux, การสื่อสารเครือข่าย, ความปลอดภัย และรูปแบบ client-server – ท้าบน VM ส่วนตัวของนักศึกษาแต่ละคนบนเครื่อง bru-rdi

User	Host	Port
labman1	10.10.180.13	2201
labman2	10.10.180.13	2202
labman3	10.10.180.13	2203
labman4	10.10.180.13	2204
labman5	10.10.180.13	2205

เป้าหมายของ lab นี้

อ่านและอธิบายผลลัพธ์จากคำสั่งตรวจสอบระบบและเครือข่ายพื้นฐานของ Linux ได้

อธิบายเส้นทางของแพ็กเก็ตจาก VM ของตัวเองออกไปอินเทอร์เน็ต (private address → NAT → gateway)

เปิด/ปิดพอร์ตด้วย firewall (ufw) และอธิบายผลที่เกิดขึ้นกับการเชื่อมต่อ

สร้างและเรียกใช้บริการ client-server อย่างง่าย พร้อมโยกกับการ SSH เข้ามาเองว่าใช้หลักการเดียวกัน (port forwarding)

ส่วนที่ 1

พื้นฐานระบบ Linux

เริ่มจากรู้จักตัวตนของเครื่องที่เพิ่งต่อเข้ามา – ใครกำลังใช้งาน เครื่องชื่ออะไร รันระบบปฏิบัติการรุ่นไหน และมีพื้นที่เก็บข้อมูลเท่าไร

1.1

ตัวตนของผู้ใช้และเครื่อง

รันคำสั่งด้านล่าง เพื่อยืนยันว่ากำลังอยู่ใน VM ของตัวเอง ไม่ใช่เครื่องแม่ (host) แล้ววางผลลัพธ์ทั้งหมดที่ได้

```
labman?@vm1
$ whoami && hostname
copy
labman4
vm1
```

1.2

รันระบบปฏิบัติการและ kernel

เช็ค distro และเวอร์ชัน kernel ที่กำลังรันอยู่

```
labman?@vm1
$ cat /etc/os-release | head -3 && uname -r
copy
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
5.15.0-190-generic
```

1.3

สิทธิ์ผู้ใช้และพื้นที่ดิสก์

ตรวจสอบ group ที่ตัวเองสังกัด และพื้นที่ดิสก์ที่เหลือใช้ได้นบน root filesystem

```
labman?@vm1
```

```
$ id && df -h /
copy
uid=1000(labman4) gid=1000(labman4) groups=1000(labman4)
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1        15G  2.2G   13G  16% /
```

คำถาม: ทำไม id ถึงไม่แสดง group sudo แต่ตอนหลัง (ส่วนที่ 3) เราจะเรียก sudo ได้แบบไม่ถามรหัสผ่าน? เดาเหตุผลไว้ก่อน แล้วย้อนมาตรวจคำตอบทีหลัง

จากผล id ที่เห็น groups=1000(labman4) ล้วนๆ – ไม่มี sudo/wheel เลย – เดาที่เป็นไปได้มากที่สุดตอนนี้คือ ข้อ 1: สิทธิ์ sudo (ถ้ามีจริง) น่าจะถูกให้แบบ per-user ผ่าน /etc/sudoers.d/ ไม่ใช่ผ่าน group membership เพราะถ้าเป็นเพราะ group ยังไม่ reload (ข้อ 2) อย่างน้อย getent group sudo ก็ควรจะโชว์ user นี้อยู่ในไฟล์ /etc/group แล้ว (แค่ session ปัจจุบันยังไม่เห็น)

ส่วนที่ 2

การสื่อสารเครือข่าย

VM ของแต่ละคนอยู่หลัง NAT ส่วนตัว (คนละวงเครือข่ายภายใน แม้จะเห็นตัวเลข IP เดียวกัน) เราจะตามรอยแพ็กเก็ตตั้งแต่ interface ของตัวเอง ไปจนถึงปลายทางบนอินเทอร์เน็ต

2.1

Interface และ IP address ของตัวเอง

ดู network interface ที่มีอยู่ และ IPv4 address ที่ได้รับ

```
labman?@vm1
$ ip -4 addr show
copy
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
default qlen 1000
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 63136sec preferred_lft 63136sec
```

คำถาม: เทียบผลลัพธ์กับเพื่อนที่ใช้ labman คนอื่น – IP ที่ได้เหมือนหรือต่างกัน? ถ้าเหมือนกัน ทำไมจึงไม่ชนกัน?

เหมือนกันได้ครับ – เพื่อนที่ใช้ VirtualBox NAT mode เหมือนกัน มีโอกาสสูงมากที่จะได้ IP 10.0.2.15 เหมือนกันเป๊ะ และไม่ชนกัน เพราะ แต่ละ VM มี network ของตัวเองแยกกันโดยสมบูรณ์

2.2

เส้นทางออกอินเทอร์เน็ต (default route)

ดู routing table เพื่อหา default gateway ที่แพ็กเก็ตทุกใบที่ไม่รู้ปลายทางจะถูกส่งไปก่อน

```
labman?@vm1
$ ip route
copy
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
10.0.2.3 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
```

คำถาม: gateway ที่เห็นคืออุปกรณ์ router จริงหรือไม่? คิดว่ามันคืออะไร

ไม่ใช่ router จริงครับ – 10.0.2.2 เป็น gateway จำลอง (virtual gateway) ที่ VirtualBox สร้างขึ้นมา เป็นซอฟต์แวร์ ไม่ใช่ฮาร์ดแวร์จริงๆที่ไหนเลย

2.3

ทดสอบการเชื่อมต่อด้วย ICMP

ทดสอบว่าปลายทางตอบกลับหรือไม่ ด้วยเลข IP ตรงๆ ก่อน แล้วค่อยด้วยชื่อโดเมน

```
labman?@vm1
$ ping -c 3 8.8.8.8
```

```

$ ping -c 3 google.com
copy
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=7.28 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=7.10 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=7.10 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 7.102/7.163/7.283/0.084 ms

PING google.com (74.125.24.138) 56(84) bytes of data.
64 bytes from sf-in-f138.1e100.net (74.125.24.138): icmp_seq=1 ttl=255 time=30.2
ms
64 bytes from sf-in-f138.1e100.net (74.125.24.138): icmp_seq=2 ttl=255 time=31.0
ms
64 bytes from sf-in-f138.1e100.net (74.125.24.138): icmp_seq=3 ttl=255 time=30.1
ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 30.110/30.448/31.023/0.408 ms

```

คำถาม: คำสั่งที่สองต้องทำงานเพิ่มอีกหนึ่งขั้นตอนก่อนจะ ping ได้ ซึ่งคำสั่งแรกไม่ต้องทำ — ขั้นตอนนั้นคืออะไร และเกี่ยวข้องกับ protocol ชื่ออะไร?

ขั้นตอนที่เพิ่มเข้ามาคือ DNS resolution (การแปลงชื่อโดเมนเป็น IP address) — เกี่ยวข้องกับ protocol ที่ชื่อ DNS (Domain Name System)

2.4

Domain Name Resolution

ก่อน query จริง ดูก่อนว่าเครื่องนี้ตั้งค่าให้ถาม DNS server ตัวไหนอยู่

```
labman?@vm1
```

```
$ resolvectl status
```

```
copy
```

```
Global
```

```
Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
```

```
resolv.conf mode: stub
```

Link 2 (enp0s3)

```
Current Scopes: DNS
```

```
Protocols: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
```

```
Current DNS Server: 10.0.2.3
```

```
DNS Servers: 10.0.2.3
```

คำถาม: DNS server ที่เห็น (บรรทัด Current DNS Server) คือ IP เดียวกับ default gateway ในข้อ 2.2 หรือไม่? ถ้าไม่ใช่ ต่างกันอย่างไร และทำหน้าที่ต่างกันตรงไหน — แก้ผังเส้นทางแพ็กเก็ตของคุณจากข้อ 2.2 ให้เพิ่มจุด DNS server แยกออกมาให้ถูกต้อง

ไม่ใช่ IP เดียวกัน

Default gateway (ข้อ 2.2): 10.0.2.2

Current DNS Server (ยืนยันจาก resolvectl status): 10.0.2.3

ต่างกันอย่างไร / หน้าที่ต่างกันตรงไหน:

10.0.2.2 (Gateway)

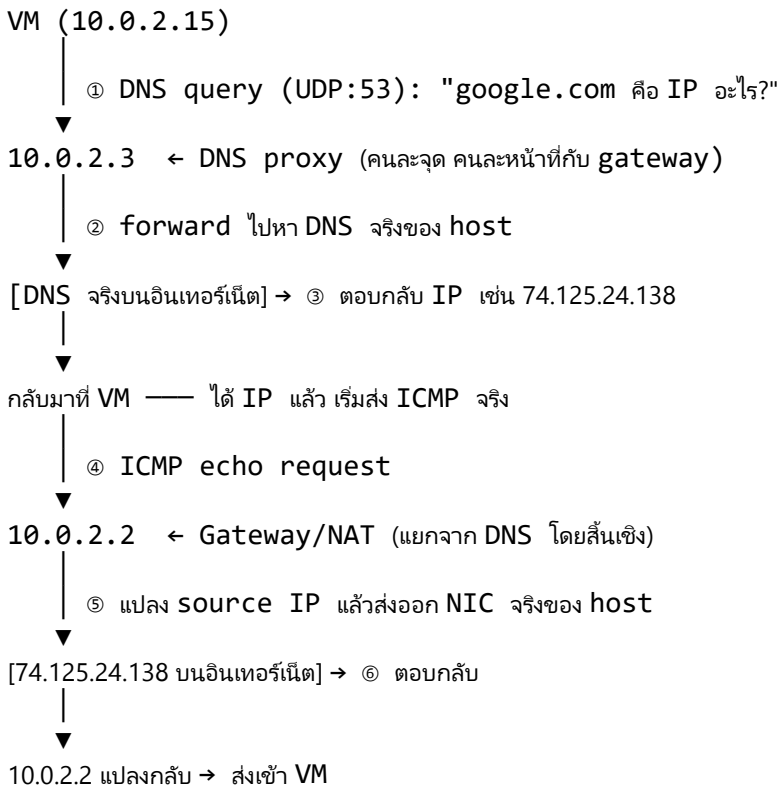
10.0.2.3 (DNS)

หน้าที่ route แพ็กเก็ตทุกชนิดออกนอก NAT โดยเฉพาะ DNS query (port 53) แปลงชื่อโดเมน → IP
ใช้กับ protocol ทุก protocol (ICMP, TCP, UDP) เฉพาะ DNS เท่านั้น
ตัวจริง NAT engine ของ VirtualBox DNS proxy/forwarder ของ VirtualBox

ทั้งคู่เป็นอุปกรณ์เสมือนที่ VirtualBox จำลองขึ้น แต่ทำหน้าที่คนละอย่าง — gateway คือ "ประตูส่งแพ็กเก็ตออก" ส่วน DNS คือ "คนตอบคำถามชื่อโดเมน" ก่อนที่แพ็กเก็ตจริงจะเริ่มเดินทาง

ผังเส้นทางที่แก้ไขให้ถูกต้อง:

กรณี ping google.com:



จุดที่ต่างจากผังเดิมในข้อ 2.2: เพิ่มขั้นตอน ①-③ (DNS resolution ผ่าน 10.0.2.3) เป็นเส้นทางแยกที่เกิดขึ้นก่อนที่ ICMP traffic (④-⑥) จะเริ่มผ่าน gateway 10.0.2.2 — สองเส้นทางนี้ไม่ทับกัน แม้จะเป็นอุปกรณ์เสมือนของ VirtualBox เหมือนกันทั้งคู่ก็ตาม

2.4b

ทดสอบ DNS query จริง

ใช้ dig เพื่อดูขั้นตอน DNS resolution ที่เกิดขึ้นเบื้องหลังก่อนคำสั่ง ping ในข้อ 2.3 จะทำงานได้

ลองเอง

```
$ dig google.com
```

```
copy
```

```
; <<>> DiG 9.18.39-0ubuntu0.22.04.6-Ubuntu <<>> google.com
```

```
;; global options: +cmd
```

```
;; Got answer:
```

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 47201
```

```
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1
```

```
;; OPT PSEUDOSECTION:
```

```
;; EDNS: version: 0, flags:; udp: 65494
```

```
;; QUESTION SECTION:
```

```

;google.com.                IN      A
;
;; ANSWER SECTION:
google.com.                 119     IN      A      74.125.24.113
google.com.                 119     IN      A      74.125.24.100
google.com.                 119     IN      A      74.125.24.138
google.com.                 119     IN      A      74.125.24.139
google.com.                 119     IN      A      74.125.24.101
google.com.                 119     IN      A      74.125.24.102

```

```

;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Sep 14 07:09:07 UTC 2026
;; MSG SIZE rcvd: 135

```

คำถาม: หา section ANSWER — IP ที่ได้ตรงกับที่ ping google.com ใช้ในข้อ 2.3 หรือไม่? บันทึก Query time เทียบกับ RTT ของ ping แล้วดูว่า SERVER ที่ dig ติดต่อ (ท้ายผลลัพธ์) ตรงกับ DNS server ที่เห็นในข้อ 2.4 หรือไม่ เขียนคำตอบ/ข้อสังเกตของคุณ...

2.5

Socket ที่กำลังเปิดรอรับการเชื่อมต่อ

ดูว่ามี service อะไรกำลัง listen บนพอร์ตไหนบ้างในเครื่องนี้

```

labman?@vm1
$ ss -tuln
copy
Netid      State      Recv-Q     Send-Q     Local
Address:Port Peer Address:Port Process
udp        UNCONN     0           0           127.0.0.53%lo:53
udp        UNCONN     0           0           10.0.2.15%enp0s3:68
tcp        LISTEN     0           4096        127.0.0.53%lo:53
tcp        LISTEN     0           128         0.0.0.0:22
tcp        LISTEN     0           128         [::]:22

```

คำถาม: พอร์ต 22 และพอร์ต 53 ที่เห็น เกี่ยวข้องกับสิ่งที่คุณกำลังทำอยู่ตอนนี้อย่างไรบ้าง?

พอร์ต 53 (DNS) — เกี่ยวข้องโดยตรงกับสิ่งที่เพิ่งวิเคราะห์ไปในข้อ 2.4

127.0.0.53%lo:53 (ทั้ง UDP และ TCP, LISTEN) คือตัว systemd-resolved กำลัง listen อยู่บน loopback — นี่คือ "stub resolver" ที่พูดถึงไปก่อนหน้านี้พอดี เวลาแอปอย่าง ping หรือ resolvectl ต้องการ resolve ชื่อโดเมน มันจะส่ง query มาที่ 127.0.0.53:53 ก่อน (ไม่ได้ยิงตรงไป 10.0.2.3 ตั้งแต่แรก) แล้ว systemd-resolved ก็จะ forward ต่อออกไปที่ 10.0.2.3 (ตามที่เห็นใน resolvectl status) อีกที นี่คือหลักฐานที่ "จับได้คาหนังคาเขา" ว่าทำไม resolv.conf mode: stub ถึงมีแบบนั้น — เพราะมันมี process จริงๆ ฟังอยู่ที่ 127.0.0.53:53 ให้เห็นเป็นรูปธรรม ไม่ใช่แค่ทฤษฎี

พอร์ต 22 (SSH) — เกี่ยวข้องกับวิธีที่คุณกำลังเชื่อมต่อเข้ามาที่ VM นี้อยู่ตอนนี้

0.0.0.0:22 และ [::]:22 (LISTEN ทั้ง IPv4/IPv6) คือ sshd กำลังรอรับการเชื่อมต่อ — นี่คือ service ที่ทำให้คุณ (หรือใครก็ตาม) สามารถ ssh เข้ามาที่ vm1 จากเครื่องอื่นได้ แปลว่า shell ที่คุณกำลังพิมพ์คำสั่งเหล่านี้อยู่ น่าจะเป็นการต่อผ่าน SSH เข้ามา (ไม่ใช่ console โดยตรงของ VirtualBox) เชื่อมกับสิ่งที่คุยไปในข้อ 2.1: ถ้า VM นี้อยู่หลัง NAT (10.0.2.15) การจะ SSH เข้ามาจากเครื่องอื่น (เช่น host) ได้ ต้องมีการ

ตั้ง port forwarding ที่ VirtualBox ไว้ล่วงหน้า (เช่น host port 2222 → VM port 22) – ถ้าคุณกำลัง SSH เข้ามาได้จริง แปลว่า port forwarding นี้ถูกตั้งไว้แล้ว
สรุปส่วนที่ 2

วาดเส้นทางแพ็กเก็ตจาก VM → gateway → อินเทอร์เน็ต ลงกระดาษ พร้อมกำกับ IP ทุกจุด

อธิบายได้ว่าทำไม VM ทุกตัวมี IP ภายในซ้ำกันแต่ไม่ชนกัน

ส่วนที่ 3

ความปลอดภัยเบื้องต้น

การเชื่อมต่อของคุณเข้ามาที่ VM นี้ ใช้กลไกความปลอดภัยอยู่แล้วโดยไม่รู้ตัว – มาดูกันว่ามันทำงานอย่างไร แล้วลองเปิด/ปิด firewall เอง

3.1

บริการที่คุณกำลังใช้อยู่ตอนนี้

ตรวจสอบสถานะของ SSH daemon ที่กำลังให้บริการ session ปัจจุบันอยู่

```
labman?@vm1
```

```
$ systemctl status ssh --no-pager
```

copy

- ssh.service - OpenBSD Secure Shell server

Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)

Active: active (running) since Mon 2026-09-14 06:46:18 UTC; 34min ago

Docs: man:sshd(8)

man:sshd_config(5)

Main PID: 3268 (sshd)

Tasks: 1 (limit: 2307)

Memory: 4.0M

CPU: 72ms

CGroup: /system.slice/ssh.service

└─3268 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

คำถาม: คุณเชื่อมต่อเข้ามาได้โดยไม่ใส่รหัสผ่านเลย – อธิบายกลไกที่ทำให้เป็นแบบนั้น (คำใบ้: ดูที่ ~/.ssh/)

กลไกคือ SSH public key authentication (แทนการ authenticate ด้วยรหัสผ่าน)

หลักการทำงาน:

มี key pair อยู่สองฝั่ง – private key อยู่ที่เครื่องที่คุณ SSH ออกมา (เช่น host หรือเครื่องที่คุณนั่งอยู่) ส่วน public key ถูกนำไปวางไว้ใน VM ที่ path:

3.2

สิทธิ์ผู้ดูแลระบบ (sudo)

เรียกสิทธิ์ root ผ่าน sudo – สังเกตว่ามีการถามรหัสผ่านหรือไม่

```
labman?@vm1
```

```
$ sudo whoami
```

copy

root

คำถาม: VM นี้ตั้งค่าไม่ให้อ่านรหัสตอน sudo เพื่อความสะดวก – ถ้าเป็นเครื่อง production จริง ความเสี่ยงของการตั้งค่าแบบนี้คืออะไร และควรแก้ด้วยวิธีไหน?

ความเสี่ยงของ NOPASSWD sudo บนเครื่อง production:

1. ลดผลกระทบจากการขโมย session/credential เหลือแค่ขั้นเดียว

ปกติมี 2 ชั้นป้องกัน: (ก) ต้องมี SSH key/credential ที่ valid ก่อน ถึงจะ login เข้ามาได้ และ (ข) ต้องรู้รหัสผ่านอีกทีถึงจะ sudo ได้ ถ้าตั้ง NOPASSWD คือถอดชั้น (ข) ออกไปเลย – ใครก็ตามที่ได้ session หรือ SSH key ของ user นี้ไป (เช่น ขโมย private key, hijack session ที่ login ค้างไว้, หรือใช้ประโยชน์จาก process ที่รันในนาม user นี้) จะ

ได้สิทธิ์ root ทันทีโดยไม่ต้องรู้อะไรเพิ่มเติมอีกเลย

2. เพิ่มความเสี่ยงจากความผิดพลาดของมนุษย์ (fat-finger)

ไม่มี "จุดหยุดคิด" ก่อนส่งคำสั่งอันตราย ปกติการต้องพิมพ์รหัสผ่านก่อน sudo (โดยเฉพาะถ้า cache หมดอายุแล้ว) เป็นจังหวะที่ทำให้คนชะงักคิดอีกทีว่า "เอ กำลังจะส่งอะไรที่มีสิทธิ์ root นะ" – พอไม่มีขั้นตอนนี้เลย คำสั่งอันตรายอย่าง rm -rf, dd, หรือแก้ไข config ผิดไฟล์ มีโอกาสหลุดออกไปง่ายขึ้น

3. เพิ่มพื้นผิวโจมตีจาก malware/script ที่รันในนาม user

ถ้ามี malicious script หรือ compromised dependency (เช่นใน CI/CD, cron job, หรือ package ที่ติดตั้งมา) รันอยู่ในนามของ user ที่มี NOPASSWD sudo มันสามารถยกระดับเป็น root ได้ง่ายๆ โดยไม่ต้องขโมยรหัสผ่านเพิ่ม – สคริปต์ automated ทั่วไปมักเรียก sudo แบบไม่มี interactive prompt อยู่แล้ว ถ้าไม่มี password gate ก็ไม่มีอะไรกันเลย

4. ไม่มี audit trail ที่ชัดเจนว่า "ตั้งใจ" ใช้สิทธิ์ root

การขอรหัสผ่านทุกครั้ง (หรือทุก timeout) สร้างจุดที่บอกได้ว่า user ตั้งใจยืนยันตัวตนเพื่อทำ privileged action จริงๆ – ถ้าไม่มีขั้นตอนนี้ การแยกแยะระหว่าง "user สั่งเอง" กับ "script/process อื่นสั่งแทน" ทำได้ยากขึ้นตอน review log

3.3

เปิดใช้งาน firewall ของตัวเอง

เปิด ufw พร้อมอนุญาตเฉพาะพอร์ต SSH ก่อนเปิดโหมด deny-by-default – สำคัญมาก ถ้าเปิด enable ก่อน allow อาจล็อกตัวเองออกจาก session ปัจจุบัน

ลองเอง

```
$ sudo ufw allow OpenSSH
```

```
$ sudo ufw enable
```

```
$ sudo ufw status verbose
```

copy

วางผลลัพธ์ที่ได้ที่นี่...

คำถาม: เปิด terminal อีกแท็บ (session ใหม่) – ยังต่อได้ปกติไหม? แล้วลอง sudo ufw deny 22/tcp จากนั้นเปิด session ใหม่อีกครั้ง เกิดอะไรขึ้น บันทึกสิ่งที่สังเกตได้ (session เดิมที่เปิดค้างจะยังไม่หลุดทันที เพราะเป็น connection ที่ established ไปแล้ว – ทำให้ถึงเป็นแบบนี้)

เขียนคำตอบ/ข้อสังเกตของคุณ...

3.4

ตรวจสอบการเข้ารหัสของ HTTPS

ดู response header ของเว็บที่ใช้ TLS เพื่อยืนยันว่าการเชื่อมต่อถูกเข้ารหัส

```
labman?@vm1
```

```
$ curl -sI https://www.google.com
```

copy

```
HTTP/2 200
```

```
content-type: text/html; charset=ISO-8859-1
```

```
content-security-policy-report-only: object-src 'none';base-uri
```

```
'self';script-src 'nonce-ybSw0iyF7DzvnHvzrnTTsg' 'strict-dynamic'
```

```
'report-sample' 'unsafe-eval' 'unsafe-inline' https: http;;report-uri
```

```
https://csp.withgoogle.com/csp/gws/other-hp
```

```
accept-ch: Sec-CH-Prefers-Color-Scheme
```

```
p3p: CP="This is not a P3P policy! See g.co/p3phelp for more info."
```

```
date: Mon, 14 Sep 2026 07:30:04 GMT
```

```
server: gws
```

```
x-xss-protection: 0
```

```
x-frame-options: SAMEORIGIN
```

```
expires: Mon, 14 Sep 2026 07:30:04 GMT
```

```
cache-control: private
```

```

set-cookie:
__Secure-STRP=ANmZwa3z6aTbbp3x6rwDlX6YB0nsMrs1NSV9xWwZgXlrvPuLDh6wewZuPXM_jg8Uzn
rt7_KyzaARu4XhgV6i99XEi-aCSETfUoY_; expires=Mon, 14-Sep-2026 07:35:04 GMT;
path=/; domain=.google.com; Secure; SameSite=strict
set-cookie: AEC=AdJVEauXNcYcnduiwYXmmmN9TTYKeN7g0ssQN0tHASBkj6_VaeLUh5a-ifo;
expires=Sat, 13-Mar-2027 07:30:04 GMT; path=/; domain=.google.com; Secure;
HttpOnly; SameSite=lax
set-cookie:
NID=534=At6bvmjFV6OUxBa_jzNHkB4uRPWZsEcEmJKiQJz2T1B_QBlp-bXV8ROwimdw1BOUMXV2bguz
b3EcaTgaB1xY5ezz3uuYhZTCwt32jFMJS5HMnSm5-eo2BSLtXODfS7N3sjqqVDWwu_u6UjxaTKUgyFsK
urymcb-03vsJYXfx9VAiGVUVPqj5M_1SYsVq5mAq0eZQplKPJcVWPS0PBx_XR1jx8eWw;
expires=Tue, 16-Mar-2027 07:30:04 GMT; path=/; domain=.google.com; HttpOnly
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000

```

คำถาม: เทียบกับ SSH ที่คุณใช้เชื่อมต่ออยู่ตอนนี้ ทั้งคู่เข้ารหัสข้อมูล แต่ต่างกันอย่างไรในแง่ protocol และวิธี handshake?
ทั้ง SSH และ HTTPS (ที่เห็นจาก header HTTP/2 200 นี้) เข้ารหัสข้อมูลเหมือนกัน แต่คนละ protocol stack และคนละวิธี handshake โดยสิ้นเชิง มาแยกดูทีละชั้น:

1. ตำแหน่งใน protocol stack

	SSH	HTTPS
Layer	การเข้ารหัส	SSH protocol เอง (มี transport layer ในตัว) TLS (แยกชั้นออกจาก HTTP)
โครงสร้าง	SSH = 1 protocol ทำทุกอย่าง (auth + encrypt + transport)	HTTP รั้งอยู่บน TLS
อีกที (TLS ทำหน้าที่เข้ารหัสอย่างเดียว, HTTP ไม่รู้เรื่องเข้ารหัสเลย)		
Port	22 (ตามที่เราเห็นใน ss ก่อนหน้า)	443 (สังเกตจาก alt-svc: h3=":443" ใน header)

ส่วนที่ 4

Client-Server และ Port Forwarding

ปิดท้ายด้วยการสร้าง server ของตัวเอง แล้วย้อนกลับมาดูว่าการที่คุณ SSH เข้ามาได้ตั้งแต่ต้น lab นี้ ก็ใช้หลักการ "แปลพอร์ต" แบบเดียวกัน

4.1

ตั้ง HTTP server ของตัวเอง

เปิด web server ง่าย ๆ ด้วย Python built-in module แล้วเรียกดูจากเครื่องเดียวกัน

```

labman?@vm1
$ echo "lab: hello from $(whoami)" > index.html
$ python3 -m http.server 8000 &
$ curl -s localhost:8000

```

copy

```
lab: hello from labman4
```

คำถาม: ในผลลัพธ์ฝั่ง server (access log) มีข้อมูลอะไรบ้าง (method, path, protocol, status code)? ระบุแต่ละส่วนจากบรรทัด log ที่ได้

จาก access log พบว่า Method คือ GET, Path คือ /index.html, Protocol คือ HTTP/1.1 และ Status Code คือ 200 ซึ่งหมายความว่า Client ส่งคำขอเพื่อเรียก /index.html ผ่าน HTTP/1.1 และ Server ประมวลผลคำขอสำเร็จ

4.1b

เปิดให้เข้าถึงจากภายนอกจริง

ระหว่างที่ python3 -m http.server 8000 ในข้อ 4.1 ยังรันอยู่ เครื่องแม่ (bru-rdi) ได้ forward พอร์ต 8000 ของ VM คุณออกสู่อินเทอร์เน็ตแล้ว ที่พอร์ต 300 ต่อท้ายด้วยเลขท้าย username ของคุณ (เช่น labman3 → 3003, labman5 → 3005) เปิด http://202.29.14.232:300X/ (แทน X ด้วยเลขของคุณ) จากอุปกรณ์ที่ไม่ได้อยู่ในวง network เดียวกับห้องเรียน (เช่น สลับมือถือไปใช้ 4G/5G)

คำถาม: เข้าถึงได้จริงหรือไม่? ระบุ URL เต็มที่ใช้ทดสอบ และอธิบายว่าทำไม IP ที่ใช้ตอนนี้ (202.29.14.232) ถึงต่างจาก IP ที่ใช้ตอน SSH ตั้งแต่ต้น lab (10.10.180.13) ทั้งที่เป็นเครื่องเดียวกัน

ถ้าผลการทดสอบ เปิดไม่ได้ ให้เขียนว่า “ไม่สามารถเข้าถึงได้จากอินเทอร์เน็ต” แทน และระบุ URL ที่ใช้ทดสอบเหมือนเดิม เพราะคำตอบ

ว่าเข้าถึงได้จริงหรือไม่ต้องอ้างอิงจากผลทดลอง

4.2

ย้อนดูว่าตัวเองเข้ามาได้อย่างไร

เทียบสิ่งที่เพิ่งทำในข้อ 4.1-4.1b กับวิธีที่คุณเชื่อมต่อเข้ามาตั้งแต่ต้น lab

คำถาม: ตอนนี้เครื่องแม่มี port forwarding คู่ขนานกันอยู่ 2 คู่ (host 220X → guest 22 สำหรับ SSH, host 300X → guest 8000 สำหรับ HTTP) อธิบายว่าทั้งสองคู่ใช้กลไกเดียวกันอย่างไร และถ้ามี labman คนที่ 6 เข้ามาอยากเปิด web server ของตัวเองบ้าง เครื่องแม่ต้องเตรียมอะไรเพิ่ม (คำใบ้: พอร์ต 8000 ฝั่ง guest ของทุกคนเข้ากันได้ไหม เพราะอะไร แล้วพอร์ตฝั่ง host ละ เข้ากันได้หรือไม่) วาดแผนภาพประกอบคำตอบ
Guest Port 8000 เข้ากันได้ เพราะอยู่คนละ VM แต่ Host Port 300X ต้องไม่ซ้ำกัน เพราะอยู่บน Host/IP เดียวกัน หากซ้ำกันจะเกิด Port Conflict และไม่สามารถกำหนดได้ว่าจะ Forward ไปยัง VM ใดครับ
สงวน

Diagram เส้นทางแพ็กเก็ต (ส่วนที่ 2)

คำตอบข้อ 3.2 และ 3.3 (firewall + sudo)

Screenshot การเข้าเว็บจากมือถือ (4G/5G) ในข้อ 4.1b

Diagram ตารางแปลพอร์ตข้อ 4.2 (SSH + HTTP คู่ขนานกัน)

Screenshot ผลลัพธ์คำสั่ง dig google.com จากข้อ 2.4b

Data Communication – VM Lab Worksheet

bru-rdi • qemu:///session • SLIRP NAT per VM

หน้าต่างนี้ยังดาวน์โหลดไม่ได้

ดาวน์โหลดไฟล์แล้วส่งผ่านช่องทางส่งงานของวิชา (เช่น Google Classroom / LMS / อีเมล)

ดาวน์โหลดไฟล์คำตอบ